Project cleanup and optimization

Completion report | 01 November 2026
WEBXPAY Next.js application

DELIVERY STATUS	**Core cleanup complete; production build and production dependency audit pass.**

Goal and completion standard

Reduce avoidable shipped media weight, defer nonessential JavaScript, improve source and dependency hygiene, and preserve the current customer-facing design and behavior. Keep the existing cleanup reports intact. Validate the final state with a clean dependency install, lint, type-check, production build, dependency audit, and public-asset reference scan.

Measured outcome

PUBLIC ASSET PAYLOAD	BEFORE	AFTER	CHANGE
Total bytes	33,544,897	21,339,365	-12,205,532
Decimal size	33.54 MB	21.34 MB	36.39% smaller
Final public files	-	255	209 source image URLs checked

The before/after figure is the public directory payload only. It excludes local generated folders such as **.next** and **node_modules**, and is not a CDN transfer or Core Web Vitals measurement.

Outcome at a glance

- Images were converted or recompressed where quality and transparency could be preserved, and all retired image URLs now redirect to existing optimized targets.
- The chatbot module is split into an on-demand chunk and loads only after the visitor activates its launcher.
- A clean Next.js production build, ESLint, TypeScript, and public image-path validation pass.
- The production dependency audit reports zero advisories. Development-only lint/tooling advisories remain documented below.

1. Delivered changes

Media payload and compatibility

Large raster assets were optimized across the site. PNG-to-WebP replacements preserve transparency where relevant; photographic JPEGs were converted or recompressed when the measured savings justified a change. Source references were updated, and 18 permanent redirects preserve the former PNG/JPEG URLs for existing links and cached pages. The compiled Next.js route manifest confirms all 18 redirects and every destination exists.

AREA	CHANGE AND RESULT
Images	Reduced public asset bytes by 12,205,532 (36.39%) versus the recorded baseline. Final inventory: 255 files / 21,339,365 bytes.
Chatbot loading	Added a lightweight launcher and dynamically imports ChatbotWidget on the first click. The build manifest confirms the widget is in a separate async chunk.
Component placement	Moved the reusable CloseIcon React component from public/ into components/solutions/icons/ and updated its consumers.
Dependency declarations	Declared the directly imported Radix visually-hidden package, aligned Next.js and its ESLint config on 15.5.28, upgraded Nodemailer to 10.3.1, and removed redundant @types/nodemailer.
Security and configuration	Removed the embedded SMTP credential fallback. Email sending now requires EMAIL_USER and EMAIL_PASS, documented in the project README.
Build tooling	Replaced deprecated next lint with eslint .; pinned the compatible PostCSS 8.5.32 override to remove all production dependency advisories without a Next.js major migration.

Behavior and release implications

The chatbot experience is intentionally deferred until the launcher is clicked; activation opens the widget immediately. SMTP delivery will fail explicitly until EMAIL_USER and EMAIL_PASS are configured in each deployment environment. Confirm those variables before release and rotate any previously deployed SMTP credential if needed.

Compatibility notes

The home page reports 183 kB First Load JS in the final Next.js build; the build summary is rounded and no comparable browser trace was collected, so no page-load reduction is claimed. The benefit verified here is deferred widget code loading, not a measured Core Web Vitals improvement.

2. Verification and handoff

CHECK	RESULT
Clean dependency install	npm ci completed successfully from the project package-lock.json.
ESLint	npm run lint passed.
TypeScript	npx tsc --noEmit passed; the production build also completed Next.js type validation.
Production build	npm run build passed on Next.js 15.5.28; all 18 static pages were generated.
Asset integrity	91 application/component/library source files scanned; 209 image URLs checked; 0 unresolved references.
URL compatibility	18 legacy image redirects compiled; all destinations resolve to files under public/.
Production dependency audit	npm audit --omit=dev: 0 info, low, moderate, high, or critical advisories.

Residual findings and limits

Development dependencies: the full npm audit reports 14 moderate advisories, confined to ESLint and related development/lint tooling. It reports no high or critical issues. The production-only audit is clean. Further tooling upgrades should be handled as a compatible, separately validated change rather than applying npm suggestions that imply disruptive version shifts.

Browser and performance testing: a live browser session, Lighthouse run, and real-user Core Web Vitals measurement were not available in this sandbox. The production build and generated manifests are the verified evidence for this report.

Asset consolidation: exact duplicates or unused public files with uncertain external consumers were not removed solely on the basis of internal source scans. This avoids silently breaking externally linked public URLs; a URL-ownership review can identify safe additional removals.

Preserved work: pre-existing worktree edits and existing cleanup reports, including WEBXPAY_PROJECT_CLEAN_STRUCTURE_PLAN.pdf, were left in place. The completion report is a separate deliverable.

RELEASE CHECKLIST	Set EMAIL_USER / EMAIL_PASS in deployment secrets; deploy the generated build; smoke-test email endpoints and the chatbot launcher; schedule a browser-based performance check and development-tool audit follow-up.